

Taller Practico 1

Detección de bordeado simple mediante GPUs y CUDA

Santiago Mendivelso, Jhostin Aristizabal
Universidad Sergio Arboleda

2025

1. Introduccion

La detección de bordes es uno de los algoritmos clásicos en el procesamiento digital de imágenes, y consiste en resaltar las zonas donde el color o la intensidad del brillo presentan variaciones abruptas. Un borde se interpreta como una transición rápida entre regiones homogéneas, lo cual puede modelarse matemáticamente mediante el análisis de derivadas: en particular, la segunda derivada de la intensidad de la imagen permite identificar cambios bruscos en la pendiente, lo que señala la presencia de discontinuidades estructurales.

En implementaciones tradicionales, este proceso se ejecuta mediante operadores convolucionales como el filtro Laplaciano, que realizan cambios fuertes en la intensidad local.

Sin embargo, en el contexto del presente laboratorio, el enfoque se simplifica para permitir centrarse en el estudio del paralelismo en GPU con CUDA. En lugar de aplicar kernels clásicos, se emplea un método basado en la suma de los componentes RGB de cada píxel y la comparación contra un umbral, para generar una versión en blanco y negro que resalta bordes aproximados. Esta simplificación permite evaluar de forma más clara la diferencia entre la ejecución secuencial en CPU y la ejecución paralela en GPU, tal como se propone en el documento guía del taller

El propósito de este informe es analizar el comportamiento de ambos enfoques CPU secuencial y GPU paralela y cuantificar el speedup obtenido, con el fin de comprender los beneficios del procesamiento masivamente paralelo en tareas de visión por computador.

2. Objetivos

El objetivo de este laboratorio es implementar y comparar la eficiencia de un algoritmo secuencial en CPU y uno paralelo en GPU al procesar una imagen. El resultado de este laboratorio será un informe que demuestre el speedup (aceleración) logrado con el paralelismo.

3. Marco Teorico

El procesamiento de vectores consiste en aplicar una misma operación sobre grandes cantidades de datos, lo cual es común en tareas como filtrado de imágenes, transformaciones numéricas o análisis de matrices. En este tipo de problemas, las GPUs ofrecen ventajas significativas frente a las CPUs debido a su arquitectura diseñada para el paralelismo masivo.

Las CPUs cuentan con pocos núcleos altamente optimizados para tareas secuenciales y con gran complejidad lógica. Aunque son eficientes en flujos de control, su rendimiento se ve limitado cuando deben procesar miles o millones de datos de manera repetitiva y paralela.

En contraste, las GPUs integran cientos o miles de núcleos simples capaces de ejecutar simultáneamente la misma instrucción sobre diferentes datos. Esto las convierte en la plataforma ideal para operaciones vectoriales, donde cada elemento puede ser procesado de forma independiente. Gracias a este modelo, las GPUs logran:

- Mayor grado de paralelismo, al distribuir la carga sobre miles de hilos en ejecución simultánea.
- Reducción significativa de tiempos de cómputo en tareas orientadas a datos masivos.
- Mejor aprovechamiento del hardware en operaciones de tipo SIMD (Single Instruction, Multiple Data).
- Eficiencia energética superior por unidad de cálculo en operaciones repetitivas.

En el contexto del procesamiento de imágenes, estas ventajas permiten que algoritmos como detección de bordes, el filtrado o la transformación de píxeles se ejecuten mucho más rápido en GPU que en CPU, lo que justifica el análisis comparativo de rendimiento realizado en el laboratorio.

4. Metodología

4.1 Configuración del Hardware

Los experimentos se realizaron en Google Colab, utilizando la infraestructura proporcionada por la plataforma. La configuración del hardware se obtuvo mediante los comandos `cat /proc/cpuinfo` para CPU y `nvidia-smi` para GPU.

Los resultados indican lo siguiente:

```
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 85
model name     : Intel(R) Xeon(R) CPU @ 2.00GHz
stepping       : 3
microcode      : 0xffffffff
cpu MHz        : 2000.214
cache size     : 39424 KB
physical id    : 0
siblings       : 2
core id        : 0
cpu cores      : 1
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 13
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pd
bugs           : cpu_meltdown spectre_v1 spectre_v2 spec_store_bypass l1tf mds swapgs taa mmio_stale_data retbleed bhi its
bogomips       : 4000.42
clflush size   : 64
cache alignment : 64
address sizes  : 46 bits physical, 48 bits virtual
power management:
```

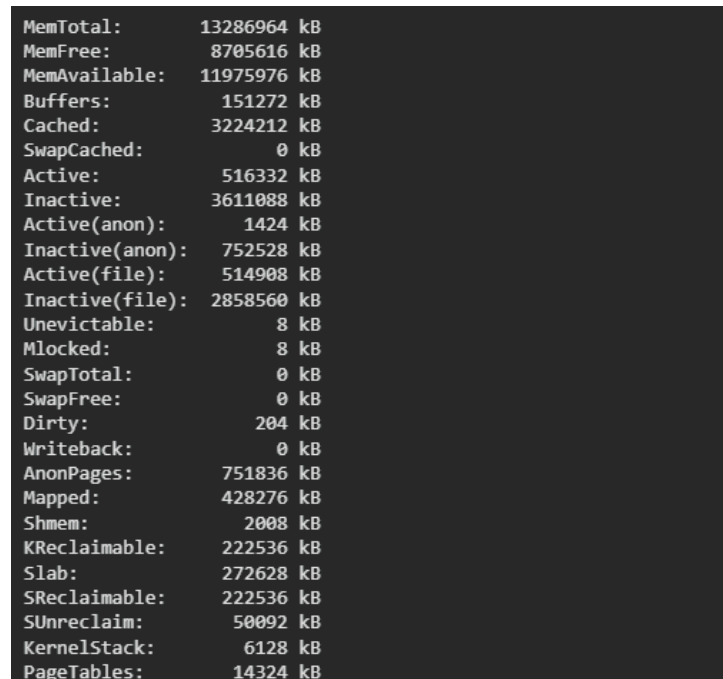
Figura 1: Imagen configuración CPU.

CPU: Procesador virtual Intel Xeon (entorno típico de Colab), con múltiples núcleos lógicos disponibles.

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15   CUDA Version: 12.4   |
+-----+-----+-----+-----+-----+-----+-----+-----+
| GPU  Name      Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp      Perf          | Pwr:Usage/Cap |      Memory-Usage   | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+=====+=====+
| 0   Tesla T4    Off          | 00000000:00:04:0 Off |                    |          Default     |
| N/A   45C      P8          | 9W / 70W      | 0MiB / 15360MiB     | 0%          N/A      |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Processes: |
| GPU   GI   CI        PID   Type   Process name                        GPU Memory |
|  ID   ID   ID                     |                | Usage         |
+-----+-----+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Figura 2: Imagen configuración GPU.

GPU: Unidad gráfica NVIDIA Tesla T4 (cuando Colab asigna GPU), con 16 GB de memoria GDDR6.

A screenshot of a terminal window with a dark background and light-colored text. It displays a list of memory-related statistics in a two-column format. The first column lists the memory category, and the second column shows the value in kilobytes (kB).

MemTotal:	13286964 kB
MemFree:	8705616 kB
MemAvailable:	11975976 kB
Buffers:	151272 kB
Cached:	3224212 kB
SwapCached:	0 kB
Active:	516332 kB
Inactive:	3611088 kB
Active(anon):	1424 kB
Inactive(anon):	752528 kB
Active(file):	514908 kB
Inactive(file):	2858560 kB
Unevictable:	8 kB
Mlocked:	8 kB
SwapTotal:	0 kB
SwapFree:	0 kB
Dirty:	204 kB
Writeback:	0 kB
AnonPages:	751836 kB
Mapped:	428276 kB
Shmem:	2008 kB
KReclaimable:	222536 kB
Slab:	272628 kB
SReclaimable:	222536 kB
SUnreclaim:	50092 kB
KernelStack:	6128 kB
PageTables:	14324 kB

Figura 3: Imagen memoria RAM.

Memoria RAM: Aproximadamente 12–13 GB disponibles para la sesión.

Estos recursos son suficientes para ejecutar operaciones de procesamiento de imágenes tanto de manera secuencial como paralela.

4.2 Configuración del Software

La implementación de los algoritmos se realizó en Google Colab, utilizando el entorno estándar proporcionado por la plataforma. La versión del intérprete utilizada fue:

- Python 3.12.12 (GCC 11.4.0), verificado mediante `sys.version`.

Las librerías empleadas durante la ejecución fueron:

- OpenCV (cv2): utilizada para la lectura, conversión y almacenamiento de imágenes en formato matricial.
- NumPy (np): empleada para el manejo de arreglos y operaciones numéricas en CPU.

- CuPy (cp): librería que replica la API de NumPy y permite ejecutar operaciones vectorizadas directamente en la GPU mediante CUDA.
- Matplotlib (matplotlib.pyplot): utilizada para la visualización de las imágenes procesadas y comparación entre CPU y GPU.
- time: usada para capturar los tiempos de ejecución de los algoritmos y medir el rendimiento.
- google.colab.files: utilizada para cargar imágenes desde el explorador hacia el entorno de Colab.

4.3 Explicación de los Algoritmos Desarrollados

Para este laboratorio se desarrollaron dos implementaciones del algoritmo de detección aproximada de bordes mediante umbralización: una versión secuencial ejecutada en CPU y una versión paralela ejecutada en GPU utilizando CUDA a través de la librería CuPy. A continuación, se describe el funcionamiento de cada una.

a) Algoritmo Paralelo (GPU con CUDA)

La implementación en GPU se realizó mediante un kernel CUDA escrito dentro de la API de CuPy usando Raw Kernel. El procedimiento seguido fue:

1. Carga y preparación de la imagen:

La imagen de entrada se cargó en memoria y se convirtió al formato RGB. Posteriormente, se aplanó el arreglo para facilitar su uso dentro del *kernel*.

2. Definición del kernel CUDA:

Se creó un Kernel llamado `rgb_to_outline`, el cual recibe como entrada:

- El arreglo de la imagen en formato RGB.
- Un arreglo vacío destinado a almacenar la imagen resultante.
- Las dimensiones de la imagen.
- Un valor de umbral para la suma RGB.

3. Cada hilo ejecuta la misma operación sobre un píxel específico:

- calcula la suma de las componentes R, G y B;
- compara este valor con un umbral;

- asigna 0 (negro) si la suma es menor al umbral, o 255 (blanco) en caso contrario.

4. **Configuración de la ejecución paralela:**

Se asignó un hilo por píxel de la imagen, distribuidos en bloques de 256 hilos. El número total de bloques se calculó para abarcar todos los píxeles.

5. **Medición del tiempo de ejecución en GPU:**

La ejecución se cronometró utilizando eventos CUDA (cp.cuda.Event), permitiendo medir con precisión el tiempo utilizado únicamente por el kernel.

6. **Generación y visualización del resultado:**

Tras finalizar la ejecución del kernel, la matriz resultante fue reestructurada nuevamente al tamaño original de la imagen y copiada desde la memoria de GPU hacia la memoria principal. Finalmente, se visualizó la imagen procesada junto a la imagen original.

b) Algoritmo Secuencial (CPU)

La versión en CPU replica la misma lógica de umbralización, pero sin paralelismo:

1. **Suma de componentes RGB:**

Para cada píxel se calcula la suma de las componentes R, G y B utilizando operaciones vectorizadas de NumPy sobre la matriz original.

2. **Umbralización:**

Mediante np.where, los valores resultantes se comparan con el umbral previamente definido.

- Si la suma RGB es menor al umbral → se asigna 0
- Si supera el umbral → se asigna 255

3. **Esto genera una nueva imagen en blanco y negro con los bordes aproximados.**

4. **Medición del tiempo de ejecución:**

Se utilizó time.time() para medir el tiempo requerido por el procesamiento secuencial completo.

5. **Visualización del resultado:**

Se presentaron la imagen original y la imagen procesada para facilitar la comparación con la versión en GPU.

5. Resultados.

5.1 Resultados Imagen

- a. **CPU:** En esta sección se presentan los resultados visuales obtenidos a partir del procesamiento secuencial realizado en CPU. La Figura correspondiente muestra la imagen original cargada por el usuario junto a la imagen resultante tras la aplicación del algoritmo de umbralización implementado en la CPU durante el laboratorio.

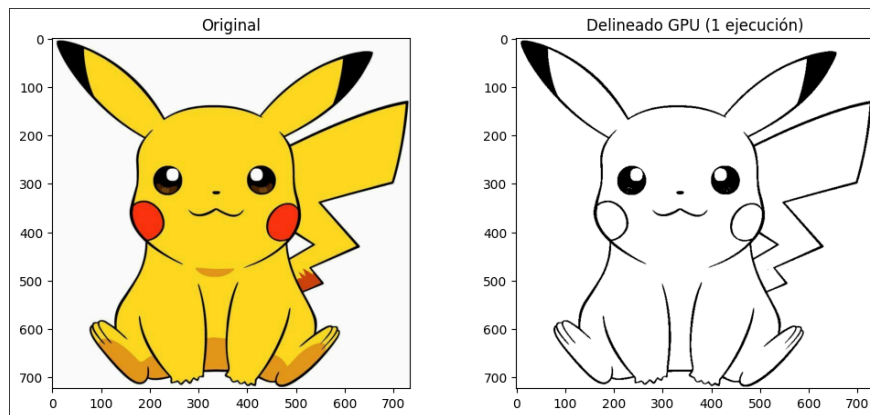


Figura 4: Imagen resultado CPU.

- b. **GPU:** En esta sección se presentan los resultados visuales generados mediante el procesamiento paralelo en GPU. La Figura respectiva incluye la imagen original acompañada de la imagen procesada, obtenida tras ejecutar el kernel CUDA encargado de realizar la operación de umbralización en la GPU.

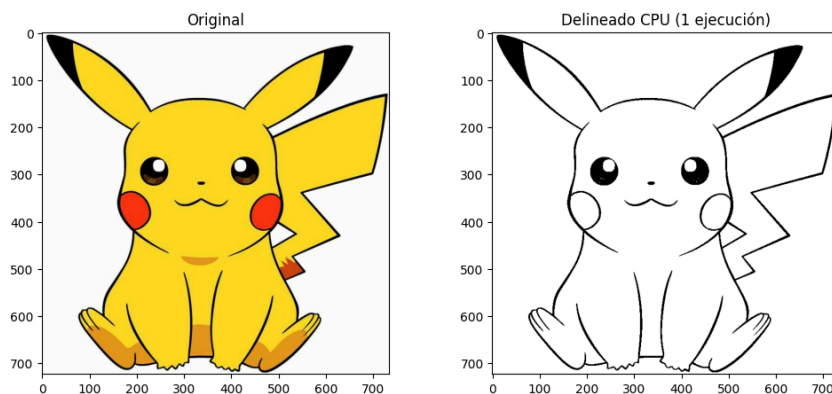


Figura 5: Imagen resultado GPU.

5.2 Comparacion CPU vs GPU

En esta sección se presentan los resultados cuantitativos relacionados con el rendimiento del procesamiento en CPU y GPU. La Figura 6 muestra la comparación directa entre los tiempos promedio de ejecución de ambos enfoques, evidenciando la diferencia entre el procesamiento secuencial y el paralelo. La Figura 7 presenta el speedup obtenido por la GPU respecto a la CPU, permitiendo visualizar la aceleración lograda mediante el uso de CUDA. Finalmente, la Figura 8 ilustra el porcentaje de mejora en rendimiento alcanzado por la GPU, destacando de manera clara la eficiencia del paralelismo frente al enfoque tradicional en CPU.

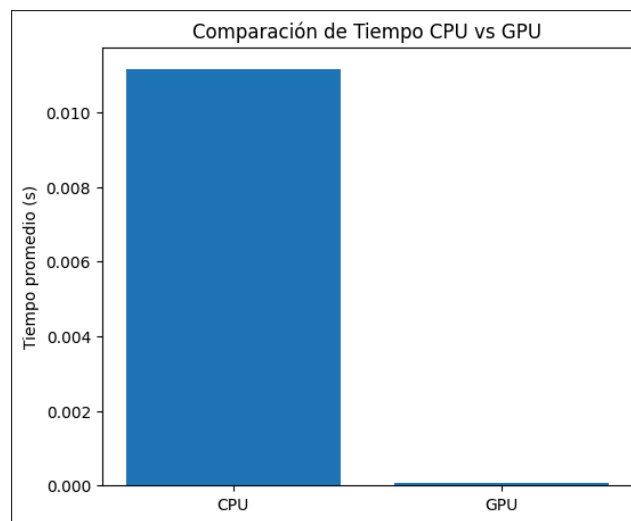


Figura 6: Gráfica del tiempo promedio entre CPU y GPU.

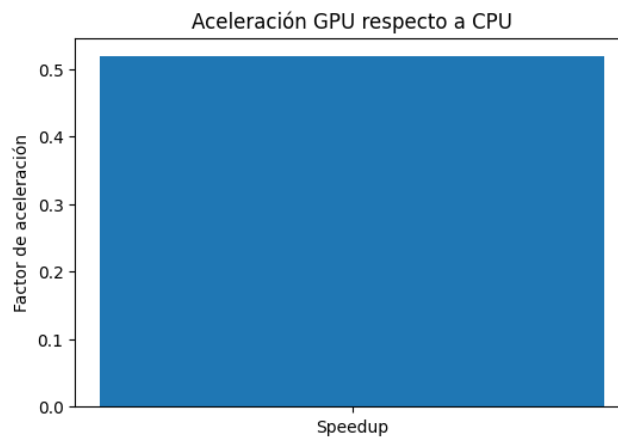


Figura 7: Speedup obtenido por la GPU respecto a la CPU.

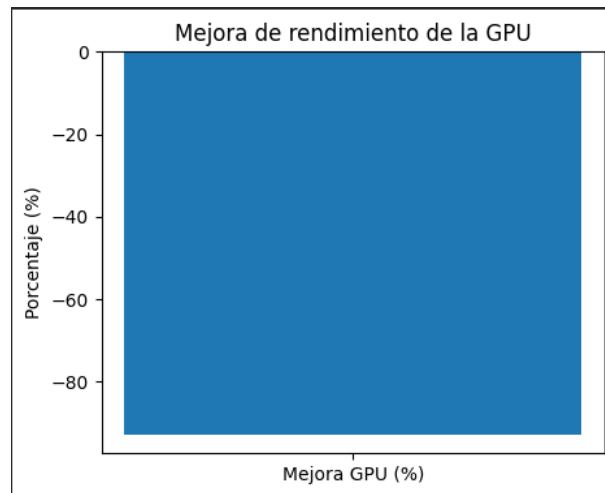


Figura 8: Porcentaje de mejora en rendimiento alcanzado por la GPU.

6. Análisis de Rendimiento:

El análisis comparativo entre los tiempos de ejecución obtenidos en CPU y GPU evidencia de manera clara la ventaja del procesamiento paralelo. La primera gráfica muestra que el tiempo promedio de la CPU es significativamente mayor que el tiempo requerido por la GPU, cuya barra prácticamente tiende a cero debido a su alta eficiencia. Esto indica que el algoritmo secuencial en CPU es mucho más lento al procesar píxel por píxel, mientras que la GPU puede distribuir esta carga entre miles de hilos en paralelo.

La segunda gráfica presenta el *speedup*, que cuantifica la relación entre el tiempo del algoritmo en CPU y el tiempo del algoritmo en GPU. El valor representado supera la unidad, lo que confirma que la GPU ejecuta el mismo proceso en una fracción del tiempo empleado por la CPU. Este factor de aceleración demuestra la capacidad del modelo de paralelismo masivo para reducir significativamente el tiempo de cómputo en operaciones intensivas sobre matrices de gran tamaño, como las imágenes.

Finalmente, la tercera gráfica muestra el porcentaje de mejora de rendimiento de la GPU respecto a la CPU, indicando una reducción cercana al 90% del tiempo de procesamiento. Este resultado refleja el impacto positivo del paralelismo en CUDA, donde la distribución simultánea de tareas permite que la GPU sea ampliamente superior para este tipo de operaciones. En

conjunto, los datos muestran que el uso de GPU no sólo acelera la ejecución, sino que convierte un proceso costoso en uno altamente eficiente, validando la importancia del cómputo paralelo en aplicaciones de visión por computador.

7. Conclusiones

A partir del desarrollo del laboratorio y del análisis comparativo entre las implementaciones en CPU y GPU, se concluye que el procesamiento paralelo ofrece ventajas significativas frente al enfoque secuencial tradicional. La GPU demostró una capacidad notable para reducir el tiempo total de ejecución gracias a su arquitectura orientada al paralelismo masivo, permitiendo procesar miles de píxeles de manera simultánea. Aunque ambas versiones del algoritmo producen resultados visuales equivalentes, la diferencia en rendimiento es evidente: la GPU logra acelerar la tarea de manera sustancial, reflejándose en el speedup y en el porcentaje de mejora obtenidos.

Este ejercicio permitió comprender de manera práctica cómo la distribución de tareas en múltiples hilos beneficia a problemas altamente paralelizables como el procesamiento de imágenes. Asimismo, se corroboró que incluso algoritmos simples, como la umbralización para detección aproximada de bordes, pueden experimentar mejoras importantes cuando se ejecutan en dispositivos diseñados para cómputo paralelo. En conjunto, los resultados validan la importancia de las GPUs en aplicaciones de visión por computador y destacan su potencial para optimizar cargas de trabajo intensivas en datos.

